

Nodos AST y Funciones Incorporadas

Motor Lógico Tipado Multidominio

1. Nodos Base

LiteralBool: true, false

LiteralInt / LiteralReal: 0, 3.14

LiteralSet: {1,2,5}, $\emptyset$

VarRef: referencia a variable x, A, p

Paren / Group: agrupación (a veces no hace falta como nodo si el parser ya arma precedencia)

2. Nodos Booleanos (conectivos)

Not: NOT ϕ

And: ϕ AND ψ

Or: ϕ OR ψ

Xor: (opcional)

Implies: (opcional, si lo querés nativo): $\phi \rightarrow \psi$

Iff / Equiv: (opcional): $\phi \leftrightarrow \psi$

3. Comparaciones Numéricas y Lógicas

Eq / Neq: $a = b$, $a \neq b$ (para números, bool, sets según definas)

Lt / Le / Gt / Ge: $<$, $\leq$, $>$, $\geq$

4. Aritmética (enteros/reales)

Add / Sub: $a + b$, $a - b$

Mul / Div: $a * b$, a / b

Neg: $-a$

Pow: (si lo soportás): a^b

Abs / Min / Max / Clamp: (opcionales pero útiles industrialmente)

5. Conjuntos (álgebra básica)

Union: $A \cup B$

Inter: $A \cap B$

Diff: $A \setminus B$

SymDiff: (opcional)

Subset / SubsetEq: $A \subset B$, $A \subseteq B$

Superset / SupersetEq: (opcional)

Member: $x \in A$

NotMember: $x \notin A$

Cardinality: $|A|$ o $\text{card}(A)$

SetBuilder / RangeSet: (si tenés): $\{i..j\}$ o $\{x \text{ in } U : \text{cond}\}$

6. Nodos Mixtos (puentes entre dominios)

BoolFromCompare: (implícito) $x+y > 10$ produce booleano

BoolFromSetPred: $x \in A$, $A \subseteq B$ producen booleano

IfThenElse: (muy valioso): if cond then a else b (para int/bool/set)

7. Funciones (tu extensión)

FuncDecl: (en IR): nombre, firma, params, cuerpo AST

FuncCall: $f(\text{arg1}, \text{arg2}, \dots)$

Return: (si tu representación lo necesita explícito)

Let/LocalBind: (opcional): variables locales dentro de función o bloque

8. Restricciones / Sistema

Constraint: un wrapper 'esto debe ser verdadero'

ConstraintList / Program: lista de constraints + tabla de símbolos + funciones

AllDifferent / GlobalConstraints: (opcionales, pero Gecode/MiniZinc usan muchas)

9. Opcional (potencia tipo CP/SMT)

QuantifierAll / Exists: sobre dominios finitos: $\forall x \text{ in } U: \dots$, $\exists \dots$

Reify: 'booleanizar' una restricción: $b \leftrightarrow (x > 5)$ o $b = (x > 5)$

Channeling: vincular set $\leftrightarrow$ bool-array (típico CP): $i \text{ in } A \leftrightarrow b[i] = \text{true}$

Funciones Incorporadas (Built-ins)

Las funciones incorporadas son aquellas que el motor conoce semánticamente, tienen evaluación directa y ayudan a la propagación y poda de restricciones.

1. Núcleo Mínimo (recomendado sí o sí)

Numéricas:

- `abs(x)`
- `min(a,b)`
- `max(a,b)`
- `pow(a,b)` (si no usás `^`)
- `clamp(x,lo,hi)` (muy útil)

Booleanas:

- `bool(x)` (cast opcional)
- `not(x)` (si no es operador)
- `and(a,b)` (opcional)
- `or(a,b)` (opcional)

Conjuntos:

- `card(A)` ■ esencial
- `empty(A)`
- `singleton(x)`
- `union(A,B)` (si no operador)
- `inter(A,B)`
- `diff(A,B)`

2. Funciones Industriales Típicas (muy usadas)

Numéricas:

- `within(x,lo,hi) →  $lo \leq x \leq hi$`
- `dist(a,b) →  $|a-b|$`
- `sign(x)`
- `round(x), floor(x), ceil(x)`

Conjuntos:

- `subset(A,B)`
- `superset(A,B)`
- `disjoint(A,B)`
- `intersects(A,B)`
- `contains(A,x)` (alias $\in$)
- `range(i,j)  $\rightarrow$  {i..j}`

Mixtas:

- `in_range(x,A)` ($x \in A$)
- `card_leq(A,k)`
- `card_geq(A,k)`
- `card_eq(A,k)`

3. Funciones CP/SMT Avanzadas (potencia solver)

Globales:

- `all_different([x1,x2,...])`
- `at_most(k, [b1,b2,...])`
- `at_least(k, [...])`
- `exactly(k, [...])`

Indexación:

- `element(i, array)  $\rightarrow$  array[i]`
- `member(x, array)`
- `index_of(x, array)`

Agregados:

- `sum(S)`
- `sum(A where cond)`
- `min_set(A)`
- `max_set(A)`

4. Funciones de Lógica Estructural

Conectan con la semántica $\{T\}, \{F\}, \{T, F\}$:

- `defined(x)` → variable asignada
- `possible(expr)` → {T,F}
- `necessarily(expr)` → {T}
- `impossible(expr)` → {F}

5. Conjuntos sobre Dominios (muy potentes)

Si el universo es discreto:

- `universe()` → U
- `complement(A)`
- `powerset(A)` (opcional)
- `choose(A)` → elemento de A
- `any(A)` → bool ($A \neq \emptyset$)

Ejemplos de Uso

- `card(A) > 3`
- `within(x,0,10)`
- `disjoint(A,B)`
- `all_different([x,y,z])`
- `element(i,Slots)`

Funciones Recomendadas para Tu Motor

Núcleo: `abs`, `min`, `max`, `card`, `empty`, `singleton`, `range`, `dist`, `within`

Conjuntos: `subset`, `superset`, `disjoint`, `intersects`

CP: `all_different`, `at_most`, `at_least`, `exactly`

Cada built-in debería tener: nombre, aridad, firma de tipos, función de evaluación y regla de propagación (opcional). Esto es lo que hace fuerte a CP.